

DAY 6 - AI Workshop

Model Evaluation & Optimization

AI Fundamentals

Metrics

Confusion Matrix

Cross-Validation

Hyperparameters

Mini Project

SESSION ROADMAP

1

Evaluation Metrics

Accuracy · Precision · Recall · F1-score

2

Confusion Matrix

Interpreting results · TP / FP / TN / FN

3

Cross-Validation

K-Fold · Stratified · Best practices

4

Hyperparameter Tuning

Grid Search · Random Search · Intuition

5

Mini Project

Group work · Dataset · Model dev & testing

Why Accuracy Alone Is Not Enough

The Fraud Detection Problem

- 100 transactions: 95 legitimate, 5 fraudulent
- A model that predicts 'NOT FRAUD' every time...
- ... achieves 95% accuracy
- **But catches ZERO actual fraud cases!**
- Accuracy hides the model's true failure

95%

accuracy

0 frauds caught

We need BETTER metrics!

The Four Core Evaluation Metrics

Accuracy

$$(TP + TN) / Total$$

Overall correct predictions. Misleading on imbalanced data.

Precision

$$TP / (TP + FP)$$

Of all positive predictions, how many were actually positive?

Recall

$$TP / (TP + FN)$$

Of all actual positives, how many did the model find?

F1-Score

$$2 \times (P \times R) / (P + R)$$

Harmonic mean of Precision & Recall. Best for imbalanced sets.

The Confusion Matrix

	Predicted: Positive	Predicted: Negative	Quick Reference
Actual: Positive	True Positive (TP) Correctly predicted POSITIVE	False Positive (FP) Predicted positive, actually negative (Type I Error)	• Precision = $TP / (TP + FP)$ • Recall = $TP / (TP + FN)$ • Accuracy = $(TP + TN) / All$ • F1 = $2 \times P \times R / (P + R)$
Actual: Negative	False Negative (FN) Predicted negative, actually positive (Type II Error)	True Negative (TN) Correctly predicted NEGATIVE	

Cross-Validation

Why Cross-Validation?

- A single train-test split can be misleading
- CV uses ALL data for both training and validation
- Gives a more reliable performance estimate
- Detects overfitting more effectively

Key Variants

- K-Fold (k=5 or 10 is standard)
- Stratified K-Fold - preserves class ratio
- Leave-One-Out (LOOCV) - for tiny datasets
- Time-Series Split - for sequential data

5-Fold Cross-Validation

TEST	Train	Train	Train	Train	Fold 1
Train	TEST	Train	Train	Train	Fold 2
Train	Train	TEST	Train	Train	Fold 3
Train	Train	Train	TEST	Train	Fold 4
Train	Train	Train	Train	TEST	Fold 5

Hyperparameter Tuning Basics

Hyperparameters are settings you choose BEFORE training — they control how the model learns.

Manual Tuning

✓ Pros

- Simple to start with
- Builds intuition

✗ Cons

- Slow, subjective
- Easy to miss good combos

Grid Search

✓ Pros

- Exhaustive — tries all combos
- Reproducible

✗ Cons

- Exponentially slow
- Wastes compute on bad areas

Random Search

✓ Pros

- Faster than Grid Search
- Covers wider space

✗ Cons

- Not guaranteed optimal
- Requires randomness control

💡 *Tip: Start with Random Search to find promising regions, then refine with Grid Search.*

MINI PROJECT

Build, Evaluate & Optimize

01 Form Groups

2–3 students per group

02 Select Dataset

From provided list or your own

03 Train Model

Apply sklearn classifier

04 Evaluate Metrics

Accuracy, Precision, Recall, F1

05 Tune & Improve

Adjust hyperparameters, re-evaluate

06 Present Findings

2-minute group summary

Python Code Reference

Metrics

```
from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, f1_score,
    confusion_matrix
)
y_pred = model.predict(X_test)
print(f1_score(y_test, y_pred))
```

Grid Search

```
from sklearn.model_selection import
    GridSearchCV

param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [3, 5, 10]
}
grid = GridSearchCV(model, param_grid, cv=5)
grid.fit(X_train, y_train)
print(grid.best_params_)
```

Cross-Validation

```
from sklearn.model_selection import
    cross_val_score, KFold

kf = KFold(n_splits=5, shuffle=True)
scores = cross_val_score(
    model, X, y, cv=kf, scoring='f1'
)
print(scores.mean())
```

Random Search

```
from sklearn.model_selection import
    RandomizedSearchCV

param_dist = {
    'n_estimators': [10,50,100,200],
    'max_depth': [3,5,10,None]
}
rs = RandomizedSearchCV(
    model, param_dist, n_iter=20, cv=5)
rs.fit(X_train, y_train)
```

Key Takeaways

- 1 Accuracy alone can be misleading - always check Precision, Recall & F1-Score.
- 2 The Confusion Matrix reveals exactly WHERE your model makes mistakes.
- 3 Cross-Validation gives a more honest estimate of real-world performance.
- 4 Hyperparameters control how a model learns - tuning them unlocks better results.
- 5 Random Search is usually the best first step for hyperparameter optimization.